Résumé Examen Java

Collections & Maps — Tout ce qu'il faut mémoriser

List · Set · Map | Comparable · Comparator | Génériques

1. Architecture globale des Collections

List	Collection	Séquence ordonnée, doublons autorisés, accès par index
Set	Collection	Unicité absolue, pas d'accès par index
Queue	Collection	File d'attente (FIFO / LIFO / Priorité)
Map	**■ PAS Collection**	**Association Clé → Valeur, clés uniques**

2. List — La Séquence

Quand utiliser :

- Quand l'**ordre d'insertion** compte.
- Quand vous avez besoin d'un accès par **index** (get(i)).
- Quand les **doublons** sont autorisés.

ArrayList	Tableau dynamique	get(i) en O(1) — lecture rapide	Insertion milieu lente O(n)
LinkedList	Liste doublement chaînée	addFirst/addLast O(1)	Parcours O(n), consomme plus de RAM
Vector	Tableau synchronisé	Thread-safe	■ OBSOLÈTE — ne pas utiliser

Exemple concret :

```
List<String> cours = new ArrayList<>();  // Declaration par interface
cours.add("Java");
cours.add("Python");
```

```
cours.add("Java");              // Doublon autorise
String premier = cours.get(0); // "Java" — acces par index O(1)
cours.remove(1);               // Supprime "Python"
// Iteration recommandee :
for (String c : cours) { System.out.println(c); }
```

Ne jamais supprimer des éléments dans une boucle for-each !
✓ *Solution : utiliser **iterator.remove()** ou **removeIf()***

3. Set — Le Filtre d'Unicité

Quand utiliser :

- Éliminer les **doublons** automatiquement.
- Vérifier rapidement si un élément **existe** (contains en O(1)).

Classe	Ordre	Null ?	Complexité	Usage idéal
HashSet	Aucun (aléatoire)	1 autorisé	O(1)	Par défaut
LinkedHashSet	Insertion conservée	1 autorisé	O(1)+	Ordre prévisible
TreeSet	Tri naturel (Comparable)	■ INTERDIT	O(log n)	Données triées

Exemple concret — déduplication d'emails :

```
List<String> emails = List.of("a@a.com","b@b.com","a@a.com");
Set<String> uniques = new HashSet<>(emails); // {a@a.com, b@b.com}

// TreeSet : tri alphabetique automatique
Set<String> trie = new TreeSet<>();
trie.add("Zoe");  trie.add("Alice");
System.out.println(trie); // [Alice, Zoe]

boolean existe = uniques.contains("a@a.com"); // true — O(1)
```

4. Map — Le Dictionnaire Clé → Valeur

La Map **N'HÉRITE PAS** de Collection. Elle associe chaque **clé unique** à exactement une valeur.

- Les **clés** sont uniques (comme un Set)
- Les **valeurs** peuvent être dupliquées

Classe	Ordre	Clé null ?	Complexité	Usage idéal
HashMap	Aucun garantie	Oui (1)	O(1)	■ Choix par défaut
LinkedHashMap	Insertion préservée	Oui (1)	O(1)+	Cache LRU
TreeMap	Tri naturel des clés	■ INTERDIT	O(log n)	Recherche par plages

Méthodes API essentielles :

`put(K, V)`	Ajoute ou remplace. Retourne l'ANCIENNE valeur (pas un boolean !)
`get(K)`	Retourne la valeur ou null si absente
`getOrDefault(K, défaut)`	Retourne la valeur ou la valeur par défaut — évite NullPointerException
`containsKey(K)`	Vérifie l'existence d'une clé — O(1)
`remove(K)`	Supprime la paire
`keySet()`	Set des clés
`values()`	Collection des valeurs
`entrySet()`	Set de Map.Entry — RECOMMANDÉ pour l'itération
`computeIfAbsent(K, fn)`	Crée la valeur si la clé n'existe pas — Java 8+

Exemple concret — comptage de mots :

```
String texte = "le chat et le chien jouent le chat";
Map<String, Integer> compteur = new HashMap<>();

for (String mot : texte.split(" ")) {
    compteur.put(mot, compteur.getOrDefault(mot, 0) + 1);
}
// Sortie : {le=3, chat=2, chien=1, et=1, jouent=1}

// ✓ Iteration via entrySet (PAS keySet + get — double hachage !)
for (Map.Entry<String, Integer> e : compteur.entrySet()) {
    System.out.println(e.getKey() + " : " + e.getValue());
}
```

1. put() retourne l'ancienne valeur, pas un boolean.
2. TreeMap + clé null = NullPointerException à l'exécution (compile mais plante !).
3. Clés immuables obligatoires (String, Integer). Modifier une clé mutable brise la table de hachage.
*4. **Itérer sur entrySet()**, jamais keySet() + get() (double hachage inutile).*

5. Génériques — Sécurité de Type

Les génériques déplacent les erreurs de type du **Runtime** vers la **Compilation**.

```
// ✗ Avant Java 5 — Raw Type dangereux
List words = new ArrayList();
words.add("Bonjour");
words.add(new Car());           // OK a la compilation...
String s = (String) words.get(1); // CRASH : ClassCastException !

// ✓ Avec Generiques — erreur detectee a la compilation
List<String> words2 = new ArrayList<>();
words2.add("Bonjour");
words2.add(42);                // Erreur compilation — parfait !
```

> Après compilation, **List<Integer>** et **List<String>** deviennent les deux **List** (type brut) dans le bytecode.
>
> *Conséquence : impossible de faire instanceof List<String> à l'exécution.*

Type Bounds — Contraindre le type générique :

```
// T doit implementer Comparable — le compilateur garantit compareTo()
public static <T extends Comparable<T>> T findMax(List<T> list) {
    T max = list.get(0);
    for (T element : list) {
        if (element.compareTo(max) > 0) max = element;
    }
    return max;
}
// Fonctionne : findMax(List.of(3, 1, 4, 1, 5)); // 5
```

Wildcards ? — Gérer l'héritage :

```
// Probleme : List<Integer> n'est PAS un List<Number> (invariance)
// Solution : utiliser le Joker ?

public double sum(List<? extends Number> list) { // Accepte Integer, Double...
    double total = 0;
    for (Number n : list) total += n.doubleValue();
    return total;
}
```

Piège	Erreur	Solution
Raw Type	List myList = new ArrayList()	Toujours spécifier le type : List<String>
Primitives	List<int> interdit	Utiliser les wrappers : List<Integer>
Soustraction	return a - b dans Comparator	Utiliser Integer.compare(a, b)

6. Comparable vs Comparator

Critère	Comparable<T>	Comparator<T>
Package	java.lang	java.util
Méthode clé	compareTo(T other)	compare(T o1, T o2)
Emplacement	Interne (modifie la classe)	Externe (classe ou lambda séparée)
Nombre d'ordres	Un seul (ordre naturel)	Plusieurs stratégies possibles
Cas d'usage	ID unique, ordre fondamental	Affichages dynamiques, tris multiples
Utilisé par	TreeMap, TreeSet, Collections.sort()	TreeMap(comp), Collections.sort(list, comp)

Exemple — Comparable (ordre naturel par ranking) :

```
public class Player implements Comparable<Player> {
    int ranking;
    public int compareTo(Player other) {
        return Integer.compare(this.ranking, other.ranking); // ✓ Jamais a-b !
    }
}
// compareTo retourne : <0 (avant) | 0 (egal) | >0 (apres)
```

Exemple — Comparator Java 8 (fluide et chainable) :

```
// Avant Java 8 (verbeux)
Collections.sort(players, new Comparator<Player>() {
    public int compare(Player p1, Player p2) {
        return Integer.compare(p1.getAge(), p2.getAge());
    }
});

// ✓ Java 8+ (declaratif)
players.sort(Comparator.comparing(Player::getAge));

// Chainer plusieurs criteres :
players.sort(Comparator.comparing(Player::getName)
            .reversed()
            .thenComparing(Player::getAge));
```

7. Arbre de Décision — Quelle structure choisir ?

Besoin de doublons ?	Non	**SET ou MAP**
Besoin de doublons ?	Oui	**LIST ou Queue**
SET : ordre important ?	Non (performance max)	**HashSet**
SET : ordre important ?	Ordre d'insertion	**LinkedHashSet**
SET : ordre important ?	Trié naturellement	**TreeSet (■ pas de null)**
MAP : ordre de l'itération ?	Peu importe	**HashMap (défaut)**

MAP : ordre de l'itération ?	Ordre d'insertion	**LinkedHashMap**
MAP : ordre de l'itération ?	Tri naturel des clés	**TreeMap (■ pas de null)**
LIST : opération dominante ?	Lecture/accès par index	**ArrayList**
LIST : opération dominante ?	Insertions aux extrémités	**LinkedList / ArrayDeque**
Multi-threading ?	Oui (lecture + écriture)	**ConcurrentHashMap**

8. Récapitulatif des Pièges Examen

■ TreeMap / TreeSet + null	Compile, mais NullPointerException à l'exécution ! Jamais de clé null dans une structure arborescente.
■ put() retourne quoi ?	L'ancienne valeur associée à la clé (ou null si c'est une nouvelle insertion), PAS un boolean.
■ Itération : keySet() vs entrySet()	• keySet() + get(k) = 2 lookups = ■ double hachage • entrySet() = 1 lookup = ✓ Toujours utiliser entrySet() !
■ Modificaton pendant for-each	Provoque ConcurrentModificationException. Utiliser iterator.remove() ou removeIf(lambda).
■ Clés immuables	Utiliser UNIQUEMENT String, Integer, etc. comme clés. Modifier un objet mutable utilisé comme clé brise le hashCode et rend la valeur inaccessible.
■ List<int> interdit	Les génériques n'acceptent pas les primitifs. Utiliser List<Integer> (autoboxing automatique).
■ return a - b dans Comparator	Integer Overflow si a très positif et b très négatif ! Utiliser Integer.compare(a, b).
■ Map n'hérite pas de Collection	Map n'a pas de méthode add(), iterator(), etc. Elle a put(), get(), entrySet()...

Bonne révision ! — Mémorisez les arbres de décision, les complexités et les pièges. Bonne chance à l'examen !